

PROJECT DOCUMENT – ADVANCED DATA ANALYTICS & ENGINEERING

Real-World Applications - Hands-on System Building

Project 1 - "AtmoSync": Micro-Climate Arbitrage Analytics

Problem Statement: *Traditional supply chain analytics rely on standard transit times and macro-weather forecasts. They fail to track how real-time, hyper-local micro-climate shifts (e.g., sudden humidity spikes inside a specific shipping container) degrade agricultural commodities before they reach the market.*

Use Case: *A commodities trader at Infotact monitors the **AtmoSync** dashboard. The streaming pipeline detects that Container A's internal temperature has drifted, increasing the spoilage rate of the avocados inside. The dashboard instantly calculates "Spoilage Arbitrage" and alerts the trader to reroute the shipment to a closer, secondary market—securing a premium price before the goods degrade below quality thresholds.*

Key Modules:

- **Streaming Ingestion (Apache Kafka):** High-throughput event streaming to ingest continuous mock IoT sensor data (temperature, humidity, vibration).
- **Cloud Data Warehouse (Snowflake):** Highly scalable storage for raw IoT streams and historical commodity pricing data.
- **Data Transformation (dbt):** ELT modeling to clean the streams and calculate complex arbitrage margins and spoilage degradation curves in SQL.
- **Visualization Layer (Apache Superset):** Real-time BI dashboard rendering live container health and financial arbitrage opportunities.

Week-wise Development Plan:

Week	Data Engineering (Kafka, Snowflake, dbt)	Analytics & Visualization (Superset)
Week 1	Ingestion Architecture: Build a Python IoT simulator generating container telemetry. Setup Kafka topics and stream data into Snowflake raw tables.	BI Foundations: Deploy Apache Superset. Connect it to the Snowflake warehouse and configure user access roles.

Week 2	ELT Pipeline: Configure dbt core. Write SQL staging models to clean raw JSON IoT payloads and join them with mock commodity pricing data.	Initial Dashboards: Build baseline charts showing historical container routes and basic temperature fluctuations.
Mid-Project Review	Pipeline Validation: Prove Kafka can handle high-frequency event streams without lag. Validate dbt staging models compile successfully.	Data Integrity Check: Ensure Superset correctly renders the raw data tables directly from Snowflake.
Week 3	Advanced Analytics: Write complex dbt models calculating the "Spoilage Arbitrage" metric (time-to-spoilage vs. distance-to-market).	Arbitrage UI: Build the core arbitrage dashboard highlighting "At-Risk" containers and recommended reroute destinations.
Week 4	Automation: Orchestrate the dbt run schedules. Optimize Snowflake query performance using clustering and materialized views.	Refine & Polish: Finalize the BI interface. Set up automated Slack/Email alerts triggered by arbitrage opportunities.
Final Project Review	A fully automated, streaming ELT pipeline handling high-velocity IoT data.	A real-time executive dashboard driving immediate financial decision-making.

Project 2 - "CogniStream": Developer Flow-State & Cognitive Load Analytics

Problem Statement: Engineering productivity is often measured using flawed metrics like "lines of code written" or "tickets closed." These measure output but ignore the friction of the development process, failing to identify what actually blocks deep, focused work.

Use Case: An Engineering Manager reviews the **CogniStream** dashboard. Instead of seeing how many commits Team A pushed, they see a "Context-Switching Tax" analysis. The data proves that Team A loses 40% of their peak cognitive flow state due to poorly timed, automated CI/CD Slack alerts interrupting their IDE sessions, allowing the manager to adjust notification policies and improve developer experience.

Key Modules:

- **Orchestration (Apache Airflow):** Scheduled DAGs pulling event logs from developer APIs (GitHub, Jira, Slack, VSCode Activity Watch).

- **OLAP Database (ClickHouse):** Columnar database heavily optimized for ingesting and querying massive volumes of timestamped event logs.
- **Data Processing (Python & Polars):** High-speed data manipulation to calculate continuous "Flow State" blocks and identify interruption triggers.
- **Custom Dashboard (React & Tremor.js):** A clean, developer-centric frontend utilizing Tremor.js for highly polished analytics components.

Week-wise Development Plan:

Week	Data Engineering (Airflow, ClickHouse, Polars)	Frontend (React, Tremor.js)
Week 1	API Ingestion: Write Python extraction scripts for mock GitHub, Slack, and IDE APIs. Set up Airflow to schedule and run these extractions daily.	UI Scaffolding: Initialize React app. Install and configure Tremor.js for enterprise-grade dashboard layouts.
Week 2	Data Modeling: Deploy ClickHouse. Use Polars to clean the extracted JSON payloads and load them into ClickHouse event tables.	Base Metrics: Build initial UI cards displaying raw metrics (e.g., total commits, hours in IDE).
Mid-Project Review	Orchestration Audit: Demonstrate successful Airflow DAG runs. Prove ClickHouse schema is optimized for time-series querying.	Component Check: Validate Tremor.js components render correctly with mock data.
Week 3	Flow-State Logic: Write advanced SQL/Polars logic to identify "Uninterrupted Flow Blocks" (e.g., 90+ mins of coding without a Slack message).	Friction UI: Build the "Context-Switching" charts, visualizing exactly when and why developers break their flow states.
Week 4	API Layer: Build a lightweight backend (e.g., FastAPI) to serve aggregated ClickHouse data securely to the React frontend.	Refine & Polish: Polish the dashboard layout. Ensure data loads quickly via the API layer.
Final Project Review	A robust orchestration engine seamlessly merging diverse, unstructured event logs into a unified OLAP schema.	A highly specialized, custom-built analytics UI measuring developer friction rather than just output.

Project 3 - "EchoChain": Circular Economy & Secondary Market Lifecycle Analytics

Problem Statement: *Manufacturers track products rigorously until the point of sale. Post-sale, the product's lifecycle becomes a data blind spot. This makes it impossible to measure true environmental impact, landfill diversion, or the potential for profitable refurbishment.*

Use Case: *A Sustainability Executive opens **EchoChain**. The system has scraped secondary market sites (like eBay) and joined that data with internal manufacturing Bills of Materials (BOM). The dashboard reveals a high "Circularity Score" for a specific laptop model—showing that while its motherboard frequently fails (warranty data), its display holds high resale value on eBay, prompting a strategic buy-back and refurbishment program.*

Key Modules:

- **Web Scraping Infrastructure (Scrapy):** Python spiders designed to extract pricing and condition data from secondary electronics markets.
- **Data Lakehouse (Databricks & Delta Lake):** Unified storage layer capable of handling both structured internal data (BOMs) and unstructured scraped web data.
- **Big Data Processing (PySpark):** Distributed computing to clean text data, perform fuzzy-matching between scraped listings and internal SKUs, and aggregate metrics.
- **Enterprise BI (PowerBI):** Corporate dashboarding tool for executive reporting and drill-down analysis.

Week-wise Development Plan:

Week	Data Engineering (Scrapy, Databricks, PySpark)	Analytics & Visualization (PowerBI)
Week 1	Data Acquisition: Build Scrapy spiders to pull mock secondary market listings. Setup Databricks workspace and initialize Delta Lake storage.	BI Setup: Connect PowerBI Desktop to Databricks SQL endpoint. Import mock internal BOM and warranty datasets.
Week 2	Lakehouse Ingestion: Land the scraped JSON/CSV data into the Bronze layer of Delta Lake. Write initial PySpark jobs to clean text and extract SKUs.	Data Modeling: Establish relationships in PowerBI between the internal manufacturing tables (BOMs, SKUs).
Mid-Project Review	Scraping Validation: Prove spiders can bypass basic blocks and extract clean pricing data. Verify Databricks cluster configuration.	Join Verification: Ensure PowerBI can successfully query the initial Delta Lake tables without timeout errors.
Week 3	Fuzzy Matching: Write complex PySpark logic to fuzzy-match messy eBay titles	Circularity Metrics: Build DAX measures in PowerBI to calculate the "Circularity

	against pristine internal SKU databases. Move data to Silver/Gold layers.	Score" and average secondary market depreciation.
Week 4	Automation: Schedule the Databricks workflows and Scrapy runs. Optimize Delta Lake tables (e.g., Z-Ordering) for faster PowerBI querying.	Executive Dashboard: Finalize the PowerBI report. Build drill-down visuals allowing executives to explore specific component lifecycles.
Final Project Review	A modern Lakehouse architecture capable of merging messy, unstructured web data with rigid internal corporate data via distributed computing.	A polished, executive-ready PowerBI report driving sustainability and secondary-market business strategies.

Common Features Across All Projects

- **The Modern Data Stack (MDS) & Cloud-Native Storage:** All three projects utilize enterprise-grade, highly scalable cloud architectures (Snowflake, Databricks Delta Lake, ClickHouse) rather than traditional on-premise relational databases, preparing you for modern data engineering roles.
- **End-to-End Pipeline Ownership:** These are not just isolated SQL or Python tasks. Every project requires you to architect the entire data lifecycle: Ingestion (Kafka/Airflow/Scrapy) $\rightarrow$ Storage/Lakehouse $\rightarrow$ Transformation (dbt/PySpark/Polars) $\rightarrow$ BI Visualization.
- **Complex, Unconventional KPIs:** Moving beyond standard retail sales or user growth dashboards, these projects require mathematical and logical rigor to calculate novel business metrics like "Spoilage Arbitrage," "Context-Switching Tax," and "Circularity Scores."
- **Diverse & Messy Data Ingestion:** You will handle a wide variety of real-world data structures, including high-velocity streaming IoT payloads, unstructured scraped web text, and nested API event logs, requiring robust data-cleaning strategies.

Overall Project Summary

Project	Target Business Domain	Technology Focus
"AtmoSync": Micro-Climate Analytics	Global Logistics & Commodities	Apache Kafka, Snowflake, dbt, Apache Superset
"CogniStream": Flow-State Analytics	Engineering Operations / HR-Tech	Apache Airflow, ClickHouse, Python (Polars), React (Tremor.js)

"EchoChain": Lifecycle Analytics	Sustainable Manufacturing & Circular Economy	Scrapy, Databricks (Delta Lake), PySpark, Microsoft PowerBI
----------------------------------	--	---

